"Antique Bar: Where Every Drink Starts with a Click!"

1. Introduction

1.1 Purpose

The purpose of this document is to provide a Software Requirements Specification (SRS) for the development of “Antique Bar”, a web application that enables users to order drinks at our bar. This document describes the requirements of the web application from the perspectives of the user, system, and customer.

1.2 Document Conventions

This document is written using the IEEE 830-1998 standard for software requirements specifications. It will include user, system, and customer requirements for the completion of the project.

1.3 Intended Audience and Reading Suggestions

This document is intended for the developers and designers working on the "Antique Bar" project. It's recommended that readers start from the beginning and read the document from start to finish to get a complete picture of the project requirements.

1.4 Product Scope

The scope of "Antique Bar" is to create a web application that allows customers to call the server to the table, servers to take orders from the customers and forward them to the barista and baristas to mark the orders as done or in process.

1.5 References

1. IEEE 830-1998: IEEE Standard for Software Requirements Specifications (IEEE, 1998).
2. Django 5.0 documentation

1.6 Overview

Antique Bar's web application streamlines table service by enabling customers to call servers, servers to take and forward orders, and baristas to update order statuses.

2. Overall Description

2.1 Product Perspective

The "Antique Bar" is a web-based application designed to optimize and streamline the customer experience and internal operations of a café or bar. It aims to bridge the gap between customers, table servers, and bar servers (bartenders) through an intuitive interface that enhances communication and service efficiency.

The application introduces a simplified ordering process where customers can easily call a server to their table with the press of a button and view the menu without needing to log in. Table servers can manage orders directly from their dashboard, while bartenders are promptly notified of incoming orders and can update their status in real time.

This system eliminates unnecessary delays in communication between staff members, ensuring a smoother workflow and improved customer satisfaction. By centralizing order management, "Antique Bar" reduces manual errors, optimizes service time, and provides the café owner with tools to manage the menu efficiently.

The application is designed to function as an independent web-based system, accessible on both desktop and mobile devices. It leverages Django for backend functionality, ensuring secure and reliable operations, while the frontend employs responsive design principles for ease of use across devices.

Through its straightforward design and integration of key functionalities, the "Antique Bar" serves as a modern solution for the traditional challenges faced in café and bar environments.

2.2 Product Functions

The "Antique Bar Ordering System" provides the following core functionalities:

1. Customer Functions
Menu Viewing: Customers can browse the menu, including items, descriptions, and prices, without requiring an account or login.
Call Server: Customers can notify a table server to attend to their table with the press of a button.

2. Table Server Functions
Order Management: Table servers can log in to their dashboard, select a specific table, and create orders based on the menu items.
Notifications: Table servers receive real-time notifications when an order is ready for pickup from the bar.

3. Bar Server (Bartender) Functions
Order Processing: Bartenders can view incoming orders in their dashboard and update their status as "In Progress" or "Ready."
Notifications: Bartenders can notify table servers when an order is ready for delivery to the customer.

4. Admin (Owner) Functions
Menu Management: Admin users can log in to add, edit, or remove items from the menu, including updating descriptions, prices, and availability.

The combination of these functions aims to create an efficient, modern, and user-focused solution for managing operations in cafés and bars.

2.3 User Classes and Characteristics

The "Antique Bar Ordering System" serves four distinct user classes, each with specific roles and access privileges:

2.3.1 Customers

Customers are the end-users of the café or bar service. They interact with the application to view the menu and request service without requiring an account. Their main characteristics include:

- Minimal interaction with the system for simplicity and ease of use.
- Ability to view menu details and call a server with a single action.

2.3.2 Table Servers

Table servers are responsible for interacting directly with customers at their tables. They use the application to manage customer orders and ensure timely service. Key characteristics include:

- Access to a dashboard for selecting tables and creating orders.
- Real-time notifications for order updates.
- Efficient tools to streamline order-taking and communication with the bar.

2.3.3 Bartenders

Bar servers manage the preparation of orders and notify table servers when they are ready for delivery. Their main characteristics include:

- A dashboard for viewing and processing incoming orders.
- The ability to update order statuses as "In Progress" or "Ready."
- Real-time communication with table servers to ensure smooth workflow.

2.3.4 Admin (Owner)

The admin user, typically the café or bar owner, has complete control over menu management and high-level oversight of the system. Key characteristics include:

- Full access to the menu for adding, editing, or removing items.
- Administrative privileges to ensure the smooth operation of the system.
- Limited day-to-day interaction, focusing primarily on periodic updates to the menu.

2.4 Operating Environment

The web application will be hosted on a web server. It will be accessible through a web browser.

2.5 Design and Implementation Constraints

The web application will be written in Python using the Django framework. It will be hosted on a web server. The data will be stored in a SQLite database.

2.6 User Documentation

No documentation will be provided to users.

2.7 Assumptions and Dependencies

The system assumes stable internet connectivity and access through modern web browsers on mobile or desktop devices. It relies on the Django framework, responsive web design, and real-time communication tools for seamless operation and synchronization across users.

3. Specific Rueqirements

3.1 Functional Requirements

3.1.1 User Requirements

3.1.1.1 View Menu

Req U1 - The web application will allow customers to view the menu.
The web application will allow customers to browse the menu, including item descriptions and prices, without requiring a login or account.

3.1.1.2 Call Server

Req U2 - The web application will allow customers to call a server to their table.
The web application will provide a "Call Server" button, enabling customers to notify a table server to attend their table.

3.1.1.3 Staff Login and Role-Based Access

Req U3 - The web application will allow staff to log in and access role-specific dashboards.
The web application will allow table servers, bar servers, and admin users to log in. Each role will have access to functionalities specific to their responsibilities.

3.1.1.4 Table Assignment for Table Servers

Req U4 - The web application will allow table servers to assign orders to tables.
Table servers can select a table from a list of active tables before creating and managing orders.

3.1.1.5 Create Orders

Req U5 - The web application will allow table servers to create orders.
Table servers can add items from the menu to an order and associate the order with a specific table.

3.1.1.6 Order Status Updates

Req U6 - The web application will allow bar servers to update order statuses.
Baristas can mark orders as "In Progress" or "Ready" and notify the corresponding table server of the update.

3.1.1.7 Notifications for Table Servers

Req U7 - The web application will notify table servers when an order is ready.
The web application will send real-time notifications to table servers when a bar server marks an order as "Ready."

3.1.1.8 Role-Specific Dashboards

Req U8 - The web application will provide dashboards for each user role.
Table servers, bar servers, and admins will have dedicated dashboards displaying their respective tasks and updates.

3.1.3 Admin Requirements

Admin requirements focus on the specific functionalities that Admins will be able to use. Admins are created by the system administrator in the Admin Dashboard. The Admins use the Django Admin interface to manage the system.

3.1.3.1 Admin Dashboard

Req A1 - The web application will provide a dashboard for admins to manage the system.
The web application will include an admin dashboard accessible only to admin users. Admins will have the ability to:

- Add, edit, and remove items from the menu, including descriptions, prices, and availability.
- Manage user roles and accounts (table servers, bar servers).

3.1.3.2 User Management

Req A2 - The web application will allow admins to manage user accounts.
Admins can:

- Create new accounts for table servers and bar servers.
- Assign or revoke specific user roles.
- Delete accounts as needed

3.2 Usability Requirements

The web application will be designed to be easy to use. The web application will be designed to be intuitive and easy to learn.

3.3 Reliability Requirements

None a this time

3.4 Performance Requirements

None a this time

3.5 Supportability Requirements

None a this time

3.6 Design Constraints

None a this time

4. System Interfaces

4.1 User Interfaces

The "Antique Bar Ordering System" will feature intuitive and responsive user interfaces tailored to each user role, ensuring a seamless experience for customers, table servers, bar servers, and admins.

4.1.1 Customer Interface

- **Menu Page**: A visually appealing, categorized menu page that displays item names, descriptions, and prices.
- **Call Server Button**: A prominently placed button on the menu page allowing customers to notify a table server.

4.1.2 Table Server Interface

- **Dashboard**: A personalized dashboard displaying:
    - A list of active tables assigned to the server.
    - Notifications for new or ready orders.
- **Order Management**: An interface for:
    - Selecting a table.
    - Browsing the menu.
    - Adding items to an order and submitting it to the bar server.
- **Notifications**: Real-time alerts when an order is marked "Ready" by a bar server.

4.1.3 Bartender Interface

- **Dashboard**: A clean interface displaying:

    - Order details, including items and assigned table.
- **Order Status Management**: Controls for marking orders as "In Progress" or "Ready."
- **Notifications**: Instant updates sent to the corresponding table server when an order is marked "Ready."

4.1.4 Admin Interface

- **Admin Dashboard**: Accessible only to admins, featuring:
    - Menu management tools for adding, editing, or removing menu items.
    - User management tools for creating, editing, or deleting user accounts and roles.

4.1.5 General Interface Features

- **Navigation Bar**: A consistent navigation bar for all user roles, including:
    - Links to relevant pages (e.g., Dashboard, Menu).
    - A "Log Out" button.

4.2 Hardware Interfaces

The application will be hosted on a secure web server running on Linux or Windows.

4.3 Software Interfaces

The application will use Django 5.0, the database will be SQLite, frontend framework will be Bootstrap.

4.4 Communications Interfaces

The application will have a secure communication interface for secure data transfer.

5. Other Nonfunctional Requirements

5.1 Security Requirements

The application will use the Django's default User management system. The application will use Django's default authentication system. The application will use Django's default authorization system. The application will use Django's default password reset system.

5.2 Safety Requirements

The application must be designed to prevent the user from performing any unsafe actions or operations.

5.3 Business Rules

The application will have business rules in place to ensure the safety and security of user data.

5.4 Quality Attributes

The application must have a high quality user interface with good usability and responsiveness. The application must also have an uptime of at least 99.

6. Other Requirements

6.1 Business Requirements

None a this time

6.2 Regulatory Requirements

None a this time